What a Token-Keyed Residual Memory Can Carry

QingGo [1]

Abstract

Sparse n-gram memory layers — DeepSeek Engram, Qwen3.8-Flash-Next and DeepSeek-V4.1-Flash — inject a retrieved vector into the residual stream at every token, and are being scaled on the premise that the memory supplies knowledge the parameters do not hold. We show that the channel is bounded before any reader is chosen: the row identifier is a deterministic function of a short addressing window, so the data-processing inequality gives $I(\text{future}; e_t \mid h_t) \leq I(\text{future}; w_t \mid h_t)$. Depth, width, linearity and training budget do not appear. The window is not the two tokens an order-3 key suggests but twelve, because a kernel-4 dilation-3 convolution follows the gate; in V4.1-Flash, which removed the convolution, it is exactly four.

Testing that prediction, three readers differing only in the corpus behind the table — Wikipedia, code and STEM — write measurably different vectors (pairwise cosine 0.38 to 0.49 on 4B) and yet produce identical output distributions: total variation 0.0000 over 1500 items on 0.8B, and on 4B a largest cross-corpus spread of one item out of 600. Removing the answer-format SFT that collapses every arm to 2.5 generated tokens reveals the other half of the bound: all four format descriptors then move outside their split-half floors, and both pre-registered directional lexical predictions hold. Suppressing the chat scaffold is content-independent; the rest of the distribution carries the corpus's surface statistics, which is the trigram prior the bound permits. Passage-conditioned recall never appears, with or without the ceiling.

Two measurements then point in different directions. A probe of the frozen table recovers about 59% of the trigram top-1 that explicit counts reach on the same corpus, and additional training makes it worse. Measured directly, the vector the reader injects has an effective dimensionality of 1.001 over 600 prompts, against 1.480 for the hidden state it is gated on — the read-out is close to a constant function of its input. The design therefore fails for the bound's reason and for a read-out defect at once, and only the first of those is unfixable by improving the reader.

1. Introduction

Sparse n-gram memory is one of the few ways to add capacity to a language model without adding parameters. DeepSeek Engram, Qwen3.8-Flash-Next and DeepSeek-V4.1-Flash hash a short token window to a row of a large disk-backed table, project that row, and inject the result into the residual stream (X. Cheng et al., 2026); Memory Grafting and XMemTransfer extend the idea to pre-training and to cross-model transfer (R. Cheng et al., 2026; OLAResearch, 2026). The premise is that the table holds associations the weights compress away.

This paper asks what such a channel can carry, and answers it without reference to the reader. The row identifier is a deterministic function of a short addressing window w_t, and the reader's contribution is a function of that row and the hidden state h_t, so the data-processing inequality bounds the memory's contribution by the window's. Depth, width, linearity and training budget do not appear, because they cannot. What is not elementary is that the window is not the one the headline key order suggests: an order-3 key reads two tokens, but the convolution that follows the gate reaches nine positions further, so the true window is twelve. V4.1-Flash removed that convolution and its window is four. The family spans a factor of three in the only quantity that bounds it, and that quantity is visible only by reading the convolution parameters. §3.2 states this precisely; §3.4 confirms it by execution rather than by arithmetic on a config.

The bound predicts that corpus content cannot reach the output, and we test that on two backbones with three readers differing only in the corpus behind the table. Under the standard recipe the three produce output distributions that agree exactly — over 1500 items per arm on 0.8B, and to within a single item in 600 on 4B — while writing visibly

[1]Independent Researcher. Correspondence to: QingGo <zyqingjohn@qq.com>.

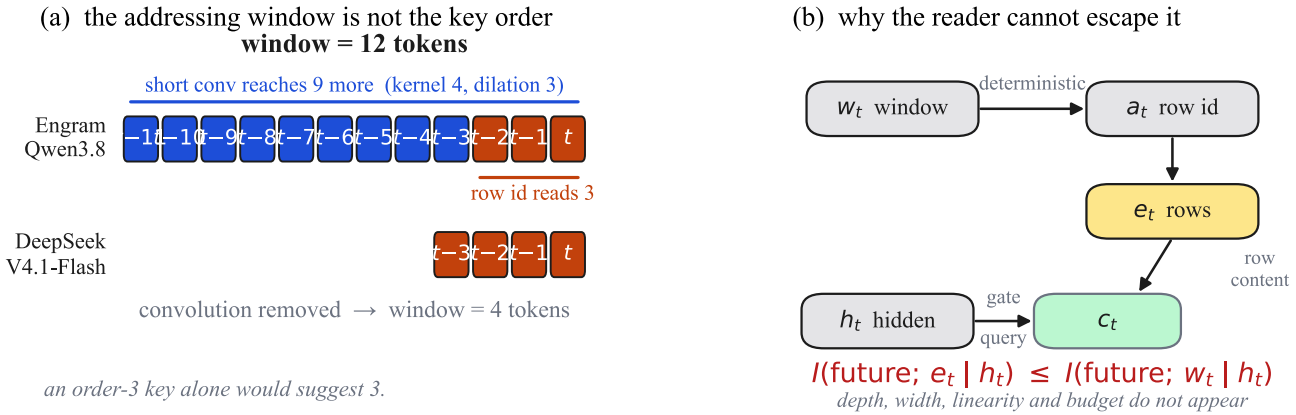

Figure 1. The channel is bounded by its addressing window, and the window is larger than the key order suggests. *(a)* An order-3 key reads two tokens, but the Engram and Qwen3.8 readers follow the gate with a kernel-4 dilation-3 depthwise convolution, so the contribution at a position draws on rows nine positions back, each of which reads two tokens further: twelve tokens, not two. V4.1-Flash removed the convolution and its window is four. *(b)* Because e_t is a deterministic function of w_t, no reader can exceed what the window already determines and h_t has not retained.

different vectors and each being best on its own corpus in a cross-evaluation. This is not a failed manipulation: the corpus changes the part of the trigram statistic the weights already encode, and under this recipe that part leaves no trace.

It would be a mistake to stop there, because every arm collapses to a two-token answer and "no format difference" could mean "no room for one". Removing the answer-format supervision changes the conclusion, and the change is the bound's other half rather than a counterexample to it: all four format descriptors move outside their split-half floors and both pre-registered directional predictions hold. Suppressing the chat scaffold is content-independent, while the rest of the distribution carries the corpus's surface statistics — which is the trigram prior the bound permits. What never appears, with or without the ceiling, is passage-conditioned recall.

Finally we ask how much of the permitted prior survives the reader. Probing the frozen table shows that it does hold a recoverable trigram prior, well above the majority floor and destroyed by shuffling, but that the read-out recovers only about 59% of what explicit counts reach. Measuring the injected vector shows why: over 600 prompts it spans effectively one dimension, against 1.480 for the hidden state it reads. The design therefore fails for the bound's reason and for a read-out defect at once, and only the first is unfixable by improving the reader. Separating them changes what the negative result is evidence for, and §4.7 is explicit about the limits of our ability to do so.

We contribute: a bound on what a token-keyed residual memory can carry, in the conditional form the gate requires, with the addressing window of each production design (twelve tokens for Engram and Qwen3.8-Flash-Next, four for V4.1-Flash); a direct test of its content prediction on two frozen backbones, where three corpus-matched readers inject different vectors and produce identical output distributions; the scoping of that null, showing that removing the answer-length ceiling reveals the corpus's surface prior exactly where the bound permits it; the measurement of the ceiling from the other side, showing that the read-out recovers about 59% of the achievable trigram top-1 and that the injected vector is nearly a constant function of its input; the falsification of the obvious repair, since longer keys do not pay at matched storage; and the audit that made all of it checkable, including one pre-registered prediction that failed.

2. Related Work

Token-keyed memory has been approached from several directions; Appendix A gives the full account. Sparse lookup layers — Engram (X. Cheng et al., 2026), Memory Layers at Scale (Berges et al., 2025), product-key memories (Lample et al., 2019) and their latent n-gram successors (Zheng et al., 2026a; 2026b) — hash a short token window to a large table and inject the result, and that is the family this paper bounds. Memory Grafting (R. Cheng et al., 2026) and XMemTransfer (OLAResearch, 2026) scale such tables or transfer them across model families. A parallel literature treats memory as an agent resource with long-range semantic recall (Borro et al., 2026; Gutiérrez et al., 2024; Packer et al., 2023; Rasmussen et al., 2025), and a third line builds non-parametric language models over a datastore (He et al., 2021; Khandelwal et al., 2020; PioneerQyw, 2026). Our results bear on the first group directly and on the third by the same argument, since a datastore addressed by a fixed

window is bounded identically; the agent-memory systems address by query rather than by window and are therefore outside the bound, which is the distinction §3.3 draws.

3. What a Token-Keyed Residual Memory Can Carry

3.1. N-Gram Addressable Memory

Let a context be a token sequence $c = (c_1, ..., c_t)$. We maintain sparse counts for each n-gram of order n:

$$p_{m(y \texttt{ mid } c)} = \frac{\text{count}(c, y)}{\sum_y} \text{count}(c, y). \tag{1}$$

The memory also returns the longest matched order and an external value index that can be audited. This memory is non-parametric, transparent, and can be rebuilt from any corpus.

3.2. The Bound

The Engram / PLE line injects a retrieved vector into the residual stream. What such a channel can carry does not depend on the reader, and because the scope of the negative results below is exactly the scope of this statement, it is worth setting out with its assumptions.

Setting. Fix a token context $c = (c_1, ..., c_t)$. Write Y for the ***future***: where we score a known continuation Y is that continuation, and where we measure emitted text it is the completion the model goes on to produce. Let w_t be the addressing window ending at t, $a_t = A(w_t)$ the row identifier, $e_t = E(a_t)$ the retrieved rows, h_t the backbone hidden state, and

$$c_t = F(h_t, e_{t-r:t}) \tag{2}$$

the contribution added to the residual stream, where r is the reach of the short causal convolution that follows the gate. Each retrieved row is keyed by an n-gram ending at its own position, so r positions of convolutional reach plus an n-token key give a window of exactly

$$|w_t| = r + n = (k-1)n + n = kn \tag{3}$$

tokens for a kernel-k convolution with dilation n. Engram and Qwen3.8-Flash-Next use $k = 4$, $n = 3$, so $|w_t| = 12$: twelve tokens, not the two an order-3 key alone implies. Section 3.4 measures this by execution rather than by reading a config.

Assumption (deterministic read-out). A, E and F are deterministic. This holds at inference in every design in the family: the only stochasticity in the graft is the sampling of Y itself, which is the quantity being predicted rather than a part of the channel. Were the read-out stochastic — inference-time dropout, or an address perturbed by noise ξ — the bound still holds conditionally, $I(Y; c_t \mid h_t, \xi) \leq I(Y; w_t \mid h_t, \xi)$, and marginalising over ξ cannot increase the left-hand side, since mutual information is convex in the channel.

Theorem. Under determinism, $I(Y; c_t \mid h_t) \leq I(Y; e_{t-r:t} \mid h_t) \leq I(Y; w_t \mid h_t)$.

Proof. Conditioned on h_t, $c_t = F(h_t, e_{t-r:t})$ is a function of $e_{t-r:t}$ alone, so $Y \to (h_t, e_{t-r:t}) \to c_t$ is a Markov chain and the data-processing inequality gives the first bound. Each row e_{t-j}, $0 \leq j \leq r$, is $E\big(A\big(w_{t-j}\big)\big)$, and w_{t-j} lies inside w_t because the convolution only reads rows whose keys fall within the same kn tokens; so $e_{t-r:t}$ is a deterministic function of w_t given h_t, and the second bound is the same inequality again. ■

Three consequences follow, and four qualifications, which matter at least as much.

Consequence 1: the bound is independent of the reader. Depth, width, linearity and training budget do not appear in it, so no reader, however expressive, can exceed it. This turns our earlier failures with hidden-state readers and direct residual injection from an empirical observation into a necessary one.

Consequence 2: it constrains addressing, not capacity. The memory can supply only what the window already determines and the hidden state has not retained, so no context-conditioned recall is possible when the determining information lies outside the window. For a question answered from a passage, the window at the answer position carries the prompt's format rather than its content: the memory may sharpen ***how*** an answer is emitted, which is the format effect we measure, but not ***which*** answer is correct.

Consequence 3: it does not say the memory is useless. Backbone weights are a lossy compression of the training corpus, and rare n-gram statistics are among what is compressed away, so the bound leaves room for exactly that. It predicts where: gains should concentrate on rare n-grams and on continuations the window determines. We test two parts of that prediction and do — knowledge probes requiring the passage return zero, and the measurable residual is a format prior rather than content — but the rarity half we cannot test, and it is worth saying why. Correlating gain with context rarity needs a count for the window n-gram, and no affordable corpus supplies one: over a 3.1M-token corpus 93% of the evaluated windows have a zero count even at the table's own addressing order of three, so a tertile split degenerates, and at the window scale essentially every twelve-gram is unique in any corpus one could count.

Qualification 1 (the bound constrains information, not effect size). The theorem bounds what the memory can tell the model about the future, not how much it can change the model's behaviour, and the two come apart sharply. A contribution that is constant — independent of w_t given h_t — satisfies $I(Y; c_t \mid h_t) = 0$ while still shifting the residual stream and therefore the output distribution. A large, reproducible behavioural effect is therefore no evidence against the bound, and the bound is no argument that an injection is harmless. This is exactly the regime §4.3 and §4.7 measure.

Qualification 2 (an address that reads the state does not escape it). Because the bound is conditional on h_t, if $a_t = A(w_t, h_t)$ then given h_t, e_t remains a function of w_t and the theorem is unchanged. What escapes the bound is addressing on information h_t has ***not*** retained — retrieval over the full context, which is the complement of a lossy summary rather than a wider view of the same neighbourhood. An earlier draft claimed any query-dependent address would relax the bound; that is wrong, and the correction matters for how the family is read.

Qualification 3 (it does not compose across positions). The theorem is per position and conditions on h_t, which is itself a function of everything injected before t. It bounds what the memory adds at a position given the state it has already produced; it does not bound what it contributes to a sequence. h_t is not a sufficient statistic for the past, so rows injected at $t' < t$ persist in h_t and can influence tokens arbitrarily far ahead: the per-position bound forbids per-position excess, not accumulation. The effect we measure is such an accumulation, and a per-position bound of zero would not have predicted it.

Qualification 4 (tightness). The bound is vacuous when $I(Y; w_t \mid h_t)$ is large, and how large it is depends on where in the prompt the window sits. On content-bearing windows it is not zero: an explicit count trigram over such a window reaches 0.2535 top-1 against a majority floor of 0.0537 (§4.6). At the answer position, where the arms actually read and where every prompt ends in the same template trigram, the addressed row is byte-identical across items, so the right-hand side is zero there and the bound is tight rather than vacuous. A zero content result at that position is therefore forced by the premise; it is not by itself evidence that a read-out could not have transmitted something, and §4.7 separates the two losses.

3.3. Scope: The Family, Not One Implementation

The scope is the family, not one implementation. DeepSeek Engram and Qwen3.8-Flash-Next address with 2- and 3-grams and keep the convolution, giving a window of twelve tokens; DeepSeek-V4.1-Flash uses 2-, 3- and 4-grams but removed the convolution, so its window is exactly four tokens and is in that respect the tightest of the three. Enlarging the table does not relax the bound, and neither does making the address depend on more of the same window.

One clarification is worth making, because it is easy to over-read. The bound is conditional on h_t, so an address that depends on the hidden state as well as the window does not escape it: if $a_t = A(w_t, h_t)$ then e_t is still a function of (w_t, h_t) given h_t, and $I(\text{future}; e_t \mid h_t) \leq I(\text{future}; w_t \mid h_t)$ is unchanged. What escapes the bound is addressing on information that h_t has **not** retained — retrieval over the full context, which is a lossy summary's complement rather than a wider view of the same neighbourhood.

3.4. The Window, Measured

The arithmetic above is a claim about the ***executed*** code path, and this project has been wrong before about what is executed — one audit item was a convolution believed absent that was in fact in the path (§7). So we measure the window rather than derive it. Build the official PLE layer from the official geometry, hold the query hidden state fixed, change a single input token at lag L, and record whether the layer's output at position t moves. The support of that response is the effective receptive field; the test asserts support, not magnitude, so it is insensitive to dtype, seed and scale. Eight geometries were checked — (k, n) in $\{(4,3),(1,4),(2,3),(4,2),(4,4),(3,3),(1,3),(2,2)\}$, spanning windows from 3 to 16 tokens — and the measured window equalled kn in every case, with every lag inside it moving the output and every lag outside it moving it by exactly zero (Appendix B). For the production geometry that is the twelve-token claim, confirmed by execution rather than by arithmetic on a config.

Two readings follow. First, the twelve-token figure is a property of Engram and Qwen3.8 and not of the family: removing the convolution collapses the window to the key order, which is why V4.1-Flash's window is four rather than three-to-four times larger. We take V4.1-Flash's design from its published description and model it as that ablation; we have not run its released weights, so that row is a statement about the design as described rather than about an implementation we measured. Second, the window is architectural: it is fixed by k and n and does not move with the embedding-table size, which is what licenses the small-configuration tests above as evidence about production geometry.

4. Does Memory Content Reach the Output?

We tested the bound's content prediction directly on two backbones. We train three readers that differ only in the corpus feeding the table — Wikipedia, code and STEM

text — and graft each into the same backbone at the same layer under identical prompts. If the memory carried corpus content, the three should answer differently.

4.1. Under the Standard Recipe, They Do Not

On Qwen3.5-0.8B in fp32, over 1500 items per arm, the answer-label distribution, the leading-surface distribution and their joint agree ***exactly***: the empirical total-variation distance is 0.0000, not merely within sampling error, and every pre-registered directional lexical prediction fails, with zero discordant pairs for code markers. Exact agreement is a stronger statement than a null result, so it is worth being precise about what it does and does not establish. It rules out any difference the sample could resolve, which for a categorical histogram over n items is $1/n$ — one item in 1500. It does not establish that the underlying distributions are identical; it establishes that the corpora are interchangeable at the resolution this evaluation has, and the equivalence margin is reported rather than assumed (see §8). On Qwen3.5-4B in bf16 — the backbone on which the format effect was first observed — the same null holds over 600 items per arm: the largest cross-corpus spread is a single item, below the split-half floor of the arm that produced it. What the corpus changes is the part of the trigram statistic the backbone weights already encode, and under this recipe that part leaves no trace in the output.

4.2. The Manipulation Reached the Injection Point

That the null is not a failed manipulation is established independently of the output. On 0.8B the three readers write measurably different vectors at the answer position (pairwise cosine similarity 0.73 to 0.76, relative L_2 distance 0.69 to 0.78, norms within a factor of 1.10); on 4B the separation is larger, cosine 0.38 to 0.49. In a 3×3 cross-evaluation each reader is best on its own corpus. The frozen tensors are bit-identical across arms and the adapter-norm ratios span only 0.886 to 1.045, well inside the threefold bound that would make the arms incomparable.

4.3. The Effect Is Large, and Larger on the Bigger Backbone

The contrast with the zero-injection arm shows the channel is not merely inert, and the effect is far larger on the 4B backbone. There, zero injection makes the model emit chat scaffolding on 95.8% of items (mean 31.2 generated tokens), while injecting a reader trained on any of the three corpora suppresses it to 0.0% (mean 2.6 tokens), with not one of the 600 items sharing its first token with the zero-injection arm ($p = 1.6 \times 10^{-173}$, McNemar exact). On 0.8B the same shift moves scaffolding from 19.5% to 0.0% and changes the first generated token on all 1500 items ($p = 1.3 \times 10^{-88}$); the reader-disabled and no-reader arms are bit-identical, so the noise floor is exactly zero rather than merely small. The graft therefore does something large and reproducible — it pushes the model off the chat template — but that shift does not depend on what the table contains.

4.4. Removing the Ceiling

This null has a scope, and testing it changes what the result means. On both backbones the injected arms saturate at 2.5 to 2.7 generated tokens, so "no format difference" could mean "no room for one". We therefore ran the regime that removes the saturation — no answer-format SFT, where generated length returns to 11 to 21 tokens — over all six arms on 600 items, so the same pre-registered rule applies without adjustment. The reader-disabled and no-reader arms are again bit-identical, giving a noise floor of exactly zero. The two halves of the rule then disagree, and the disagreement is the result. The coarse label taxonomy still reports a perturbation artifact, because the scaffold rate spans only 0.0100 against a threshold of 0.0122; every finer format descriptor reports content dependence. That division is why the co-primary descriptors were pre-registered alongside the taxonomy, on the stated grounds that the taxonomy can be insensitive to a real format shift. All four move outside each arm's own split-half sampling floor (leading-surface total variation 0.1433 against a floor of 0.0237; first-token 0.4983 against 0.2176), and both pre-registered directional lexical predictions hold: the code reader emits more code markers ($+0.0333$, one-sided $p = 3.9 \times 10^{-4}$) and the Wikipedia reader emits far more prose markers ($+0.6933$, $p = 4.5 \times 10^{-34}$), where under saturation both predictions had failed outright.

The rule, its thresholds and the co-primary descriptors were fixed in the repository before the six-arm run, and the analysis script that scores the unsaturated regime is the same one that scored the saturated regime, applied unchanged. All three directional predictions were pre-registered and all three are reported, including the one that fails in both regimes (math markers, $p = 0.16$ unsaturated, 0.84 saturated). We apply no multiple-comparison correction across the four co-primary descriptors, and instead report each against its own arm's split-half floor, which is the stricter comparison for a single arm; across the three directional predictions we report uncorrected p values, and note that a Bonferroni correction at $0.05/3$ would leave both positive results significant.

4.5. The Layering Is What the Bound Predicts

One point of setup has to be explicit, because it scopes the claim. Our tables are general corpus tables — built from Wikipedia, code and STEM text — and not from the evaluation passages. A table therefore cannot recall a passage it was never built from, and the zero we report is

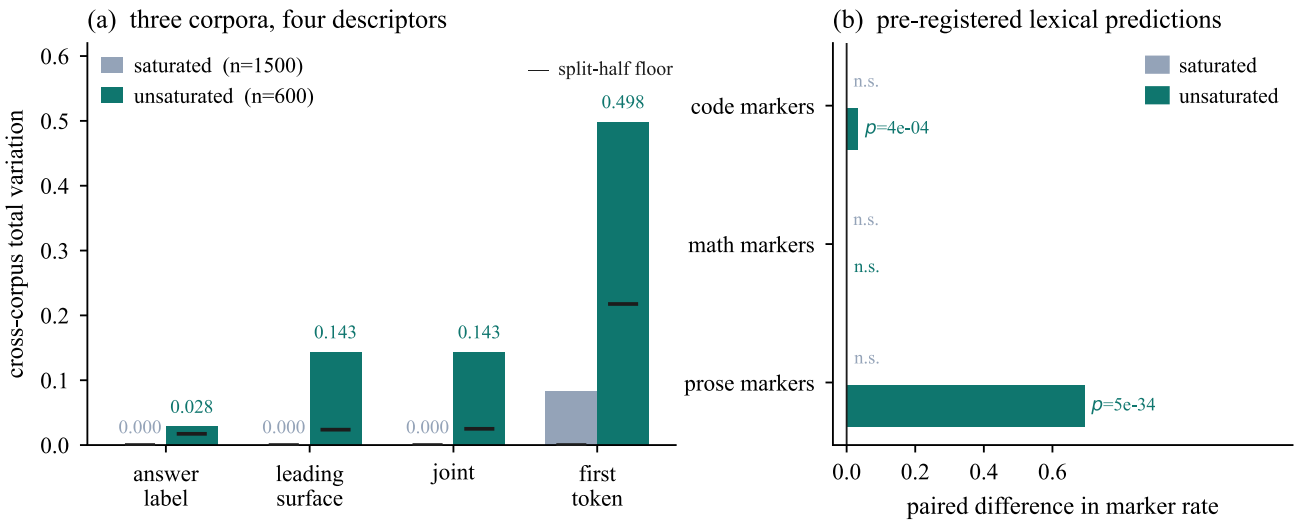

Figure 2. Removing the answer-length ceiling reveals content dependence that the saturated regime could not show. *(a)* Cross-corpus total variation for four format descriptors, against each arm's own split-half floor (short ticks). Under the standard recipe three descriptors are pinned at exactly zero; with the ceiling removed all four move outside their floors. *(b)* The pre-registered directional lexical predictions. Both were refuted under saturation and both hold without it. Math markers remain non-significant in both regimes, which is the expected result for a contrast the corpus does not separate.

not evidence that a table built from the passage would fail. What the bound forbids is narrower and does not depend on that: at the answer position the window holds the prompt's format rather than its content, so passage content cannot be supplied through this channel whatever the table contains. The experiment that would separate the two readings — a table built from the evaluation passages, with a question whose answer lies outside the window — is not in this paper, and it is the first thing we would add. We also note that TriviaQA is a weak probe of this: the frozen base model is already at 0.005 exact match, so the task cannot detect an improvement even if one existed, and a task where the base model is measurably above chance is required.

So the correct statement is layered rather than absolute, and it is the layering the bound predicts. Suppressing the chat scaffold is content-independent — all three readers suppress it, and they differ from each other by at most 0.01. The rest of the output distribution is content-dependent, and that dependence **is** the trigram prior the bound allows the channel to carry: a reader trained on Wikipedia supplies Wikipedia-like surface statistics, one trained on code supplies code-like ones. What never appears, with or without the ceiling, is passage-conditioned recall — the knowledge probes that require the passage return zero. The channel carries the corpus prior and nothing above it.

4.6. How Much Prior Is Actually There

The bound permits the memory to carry a corpus's rare n-gram statistics, so the useful question is how much of that prior the frozen table actually holds and how much of it the read-out recovers. We probe the raw 2560-dimensional rows directly, on held-out windows, against explicit counts on the same corpus as a reference frame. Majority-class prediction gives a floor of 0.0537 top-1. A count bigram reaches 0.2264 and a count trigram 0.2535. A ridge probe on the frozen rows reaches 0.1280; an MLP reaches 0.1502 at its best epoch and 0.1225 at 24 epochs, so additional training makes it worse rather than better. Shuffled-row controls collapse below the majority floor at 0.0459 and 0.0373. The table therefore does hold a recoverable trigram prior — well above the floor and destroyed by shuffling — but the read-out recovers only about 59% of what explicit counts reach.

4.7. The Read-Out Does Not Transmit It

That gap is not a small correction, and measuring the injected vector directly shows how large it is. Over 600 diverse prompts the vector the reader adds to the residual stream has an effective dimensionality of 1.001: 99.93% of its variance lies in a single direction, and two prompts are separated by about two degrees. The hidden state the reader is gated on sits at 1.480, with 81.9% in its leading direction. A generic linear map does not do this — three random maps of the same shape leave the hidden state's ratio at 1.454, 1.472 and 1.485 — and replacing the retrieved rows with permuted ones, at the same checkpoint, moves the injected direction by only about five degrees.

The measurement is 600 items at the same layer and dtype as the arms. The participation ratio is $(\sum \lambda)^2 / \sum \lambda^2$ over the eigenvalues of the covariance of the 600 injected

vectors, with 95% intervals from 2000 bootstrap resamples. It gives 1.0014 $[1.0012, 1.0015]$ for c_t against 1.480 $[1.433, 1.535]$ for h_t, so the two are separated by far more than resampling error. The injected vector is also close to orthogonal to the state that produced it (mean cosine 0.04) and its norm is $0.53 + -\!-0.33$ against $1.33 + -\!-0.06$ for that state — a mean ratio of 0.40 with a coefficient of variation of 0.62. The collapse is therefore not a shut gate: the injection is comparable in magnitude to the state it modifies, and the gate opens onto the same direction every time.

Two explanations remain, and they are separable. The reader's arithmetic is $e_t \to$ value proj $\to$ branch sum $\to c_t$, and measuring it at positions further back varies its input. At the answer position the addressed row is not merely similar across items but byte-identical: all 200 items of each task address the same row, because every prompt ends in the same template trigram, so the bound's right-hand side is zero there and the null is forced by its premise. Twelve tokens back, the same read-out is fed a genuinely item-specific row — 190 of 200 distinct, effective dimensionality 136.7 — and still emits 1.02, discarding about 99.3% of the effective dimensionality available to it: a factor of ten in the gate and value path, a further factor of three in the output projection. Appendix J gives the ladder and the figure. This also corrects the argument above, since the random-linear-map control maps the hidden state and never the addressed row.

4.8. The Obvious Repair Does Not Pay

Nor does lengthening the key. Replacing the 4-gram address with 8-token, 16-token and longest-suffix memories at matched storage ties or loses at every budget and training size on both corpora, so the short window costs nothing measurable. That the window can be widened without gain, while the corpus the table is built from cannot be changed without gain either, is the same statement twice: the channel's output is fixed by what the window already determines.

5. The Logit-Correction Interface Under the Same Bound

Four terms recur and we use them in one sense throughout. The ***reader*** is the module that turns retrieved rows into a residual contribution; its ***read-out*** is the arithmetic inside it — the projections, the gate and the short convolution — and is what our probes measure. The ***projector*** is the logit-level analogue that produces (α_t, β_t), and an ***adapter*** is a parametric update to the backbone. Reader and projector are two interfaces to the memory; the adapter is not a memory interface at all.

The second interface replaces residual injection with a per-token logit correction, $\log p_{\text{fused}}(y) = \log p_{b(y)} + \alpha_t \log p_{m(y)} + \beta_t$, gated by a learned token policy. It is not an exemption: logit-level fusion of an n-gram memory is **also** a function of the addressing window, so the bound applies exactly as it does to the residual graft, and the permitted prior is the same in both cases. What the projector gets is a better interface, not a larger allowance. The graft loses that prior twice — once through the read-out, and again through a residual bottleneck — while the logit correction loses it once, and spends it in the basis where it is used. Appendix C gives the method and the algorithm listings.

The gain is confined to low-entropy local continuation — the region the bound leaves open — and scales with projector training data: $+0.1044$ nats over 100 samples (95% CI $[0.0251, 0.1866]$), $+0.2249$ at 10k (CI $[0.1436, 0.3255]$), all five seeds positive. Beyond that the joint system separates by resource rather than by strength (Appendix D): adding PLE to the best retrieval-plus-adapter combination changes nothing on knowledge or code-output while costing 0.123 nats on arithmetic. On HumanEval neither ordering survives sampling. A token-level n-gram prior sharpens the base model's local mode rather than adding capability.

6. Analysis

The channel helps where the true next token is highly predictable from local n-grams — code, identifiers, repeated structural patterns, name-like continuations — which is exactly the region the bound leaves open, and it fails where the answer needs world knowledge or long-range reasoning, which is why TriviaQA sits at 0.005 and why unconditional fusion degrades open-ended generation without the policy. For token-keyed memory that failure is necessary rather than contingent.

Two mechanisms escape the bound, and only two: retrieval, which addresses by query rather than by a fixed window, and parametric adapters, which are not a memory channel at all. PLE, RAG and adapters are therefore not three points on one axis of strength but three interfaces to three resources, and only the first is bounded as this paper describes. That reading holds against our own numbers: the projector's gain is confined to low-entropy local continuation and is absent on knowledge tasks.

7. Method Audit

A channel must be checked against its information-theoretic ceiling before its capacity is scaled, and a memory module must be evaluated by whether it uses memory content rather than by whether loss improves. Earlier hidden-state readers,

MLP readers and direct residual injection often improved loss-like metrics and failed the real-versus-control test, and the bound explains why: a reader translating a token-keyed row into the residual stream cannot carry context the window does not determine.

The less comfortable lesson is that a claim is only as trustworthy as the check behind it. Table 7 lists every case we found in which a change was made, a claim written, and the falsifying check either absent or silently skipped, with the full table in Appendix G. None was found by re-reading code; each was found by making the failure detectable — invariant assertions, regression tests that fail on the specific bug, a flag that converts a silent skip into a failure, and manifests recording a fingerprint of what was measured. Three of these were failures of the pipeline's **account of itself** rather than of the experiment — the arm count, the items scored, and an offline reproduction that averaged over a matrix diagonal — and in each the model's numbers were right while our description of our own artifacts was wrong. That asymmetry is why this paper reports an equivalence margin rather than a claim of identical distributions.

Finally, a failure that is not a mistake. Our last experiment pre-registered a prediction — that the injected vector would be more nearly constant for a task whose window is mostly a fixed instruction suffix — and it was wrong: within-task similarity came out at 0.9994 and 0.9992, a difference of $+0.0002$ that a permutation test over task labels declined to distinguish from noise ($p = 0.27$). Fixing the statistic and its refutation condition in advance made the wrong result legible, and it pointed straight at the near-constant read-out, which is a stronger finding than the prediction would have been.

8. Limitations

The code results are directional rather than benchmark-scale, and the unsaturated regime that reveals the corpus's surface prior is 600 items on one backbone under a different training recipe; extending it to more backbones and larger samples is the most direct further test of the layering result, and we have not done it. The bound's rarity prediction is stated but not tested, and §3.2 shows it is not testable by corpus counting at this window scale. The conclusion is scoped to frozen-table grafts on a frozen backbone at a single mid-stack layer with a 0.8B–4B target, so co-training, learned routing, adaptive layer placement, the official layer-1 placement, and deployment trade-offs in table size and memory bandwidth are untested rather than refuted. HumanEval is public and we cannot rule out contamination of the base model's training data, and the equivalence result in §4.1 is exact agreement of empirical histograms at the sample's $1/n$ resolution rather than proof that the underlying distributions are identical. The read-out collapse is localised in §4.7, but that localisation rests on one reader checkpoint at one layer. Appendix H lists these and the remaining limitations in full.

9. Conclusion

We asked what a token-keyed residual memory can carry, and the answer is set before any reader is chosen. The row identifier is a deterministic function of a short addressing window, so the memory's contribution cannot exceed what that window already determines and the hidden state has not retained. In the Engram and Qwen3.8-Flash-Next designs that window is twelve tokens rather than the two an order-3 key suggests; in V4.1-Flash, which removed the convolution, it is four.

The prediction this makes about content survives a direct test. Three readers differing only in the corpus behind the table write measurably different vectors and, under the standard recipe, produce output distributions that agree exactly. Removing the answer-length ceiling completes the picture rather than breaking the bound: the corpus's surface statistics then appear, which is the trigram prior the channel is permitted to carry, while passage-conditioned recall remains at zero.

Two measurements size what remains, and they are now separable. The read-out recovers about 59% of the trigram top-1 that explicit counts reach, and at the answer position the addressed row is byte-identical across items, so the null there is forced by the premise rather than by any defect. Twelve tokens back the same read-out collapses an input of effective dimensionality 137 to 1.02, so the read-out defect is real as well, and only it is fixable by improving the reader. Lengthening the key does not pay at matched storage on any corpus or budget we tested. What remains is the complement — retrieval, which addresses by query, and parametric adapters, which are not a memory channel — together with the read-out itself.

References

Asai, A., Wu, Z., Wang, Y., Sil, A., and Hajishirzi, H. *Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection*, 2023. https://arxiv.org/abs/2310.11511v1

Asl, M. A., Asgari-Bidhendi, M., and Minaei-Bidgoli, B. *FAIR-RAG: Faithful Adaptive Iterative Refinement for Retrieval-Augmented Generation*, 2025. https://arxiv.org/abs/2510.22344v1

Authors, R. *RAGRouter-Bench: Benchmarks for Learned Query Routing in Retrieval-Augmented Generation*, 2026. https://ar5iv.labs.arxiv.org/html/2602.00296

Authors, T. *TokenMem: Faithful Knowledge Injection for Frozen LLMs*, 2026. https://arxiv.org/html/2607.22625

Bai, Y., Lv, X., Zhang, J., and others. *LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding*, 2024. https://arxiv.org/abs/2308.14508

Berges, V.-P., Oguz, B., Haziza, D., and others. Memory Layers at Scale. *International Conference on Machine Learning*, 2025. https://mlanthology.org/icml/2025/berges2025icml-memory/

Bhardwaj, R., Vaddamanu, S., and others. kNN-LM Does Not Improve Open-ended Text Generation. *Proceedings of EMNLP*, 2023. https://aclanthology.org/2023.emnlp-main.929/

Borro, L. C., Macarini, L. A. B., Tindall, G., Montero, M., and Struck, A. B. *Memori: A Persistent Memory Layer for Efficient, Context-Aware LLM Agents*, 2026. https://arxiv.org/abs/2603.19935v1

Cheng, R. et al. *Memory Grafting: Scaling Language Model Pre-training via Offline Conditional Memory*, 2026. https://papers.cool/arxiv/2605.20948

Cheng, X. et al. *Conditional Memory via Scalable Lookup: A New Axis of Sparsity for Large Language Models*, 2026. https://github.com/deepseek-ai/Engram

Das, P., Ko, C.-Y., Dai, S., Kollias, G., Chaudhury, S., and Lozano, A. *Can Memory-Augmented Language Models Generalize on Reasoning-in-a-Haystack Tasks?*, 2025. https://arxiv.org/abs/2503.07903v1

Dettmers, T., Pagnoni, A., Holtzman, A., and Zettlemoyer, L. QLoRA: Efficient Finetuning of Quantized LLMs. *Advances in Neural Information Processing Systems*, 2023.

Gros, D., and Devanbu, P. *Localized Calibrated Uncertainty in Code Language Models*, 2025. https://arxiv.org/abs/2512.24560v1

Group, L. *MemSFT: Mitigating Alignment Tax with an External Parametric Memory*, 2026. https://github.com/LUMIA-Group/MemSFT

Gutiérrez, B. J., Shu, Y., Gu, Y., Yasunaga, M., and Su, Y. *HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models*, 2024. https://arxiv.org/abs/2405.14831v3

Gutiérrez, B. J., Shu, Y., Qi, W., Zhou, S., and Su, Y. *From RAG to Memory: Non-Parametric Continual Learning for Large Language Models*, 2025. https://arxiv.org/abs/2502.14802v2

He, J., Neubig, G., and Berg-Kirkpatrick, T. *Efficient Nearest Neighbor Language Models*, 2021. https://arxiv.org/abs/2109.04212v3

Hu, E. J., Shen, Y., Wallis, P., and others. LoRA: Low-Rank Adaptation of Large Language Models. *International Conference on Learning Representations*, 2022.

Huang, Y., Liu, Y., Zhao, R., Zhong, X., Yue, X., and Jiang, L. *MemOrb: A Plug-and-Play Verbal-Reinforcement Memory Layer for E-Commerce Customer Service*, 2025. https://arxiv.org/abs/2509.18713v1

Hwang, J., Park, J., Park, H., Kim, D., Park, S., and Ok, J. *Retrieval-Augmented Generation with Estimation of Source Reliability*, 2024. https://arxiv.org/abs/2410.22954v5

Jiang, J., Wu, X., Song, W., Yang, B., Zhou, Y., Ge, H., Lee, H. P., Liang, Y., and Wu, C. *ReliableRAG: Combating Misinformation in Retrieval-Augmented Generation via Reliability-Guided Reasoning Chains*, 2026. https://arxiv.org/abs/2608.25487v1

Jiang, T., Huang, S., Luo, S., and others. *MoRA: High-Rank Updating for Parameter-Efficient Fine-Tuning*, 2024. https://arxiv.org/abs/2405.12130

Jin, M., Luo, W., Cheng, S., Wang, X., Hua, W., Tang, R., Wang, W. Y., and Zhang, Y. *Disentangling Memory and Reasoning Ability in Large Language Models*, 2024. https://arxiv.org/abs/2411.13504v3

Khandelwal, U., Levy, O., Jurafsky, D., Zettlemoyer, L., and Lewis, M. Generalization through Memorization: Nearest Neighbor Language Models. *International Conference on Learning Representations*, 2020. https://papers.lunadong.com/paper/4555

Kim, H., Kim, Y. J., and Bak, J. *PEMA: An Offsite-Tunable Plug-in External Memory Adaptation for Language Models*, 2023. https://arxiv.org/abs/2311.08590v3

Lample, G., Sablayrolles, A., Ranzato, M., Denoyer, L., and Jégou, H. *Large Memory Layers with Product Keys*, 2019. https://arxiv.org/abs/1907.05242v2

Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., and Liang, P. *Lost in the Middle: How Language Models Use Long Contexts*, 2023. https://arxiv.org/abs/2307.03172v3

Lyu, Q., Shridhar, K., Malaviya, C., Zhang, L., Elazar, Y., Tandon, N., Apidianaki, M., Sachan, M., and Callison-Burch, C. *Calibrating Large Language Models with*

Sample Consistency, 2024. https://arxiv.org/abs/2402.13904v2

Miao, M. M., and Ungar, L. *Closing the Confidence-Faithfulness Gap in Large Language Models*, 2026. https://arxiv.org/abs/2603.25052v2

Nelson, E., Kollias, G., Das, P., Chaudhury, S., and Dan, S. *Needle in the Haystack for Memory Based Large Language Models*, 2024. https://arxiv.org/abs/2407.01437v2

OLAResearch. *Cross-Model Memory Transfer via Target-Side Reader Adaptation*, 2026. https://github.com/OLAResearch/XMemTransfer

Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., and Gonzalez, J. E. *MemGPT: Towards LLMs as Operating Systems*, 2023. https://arxiv.org/abs/2310.08560v2

Pan, X., Yao, W., Zhang, H., Yu, D., Yu, D., and Chen, J. *Knowledge-in-Context: Towards Knowledgeable Semi-Parametric Language Models*, 2022. https://arxiv.org/abs/2210.16433v3

PioneerQyw. *NGM: A Plug-and-Play Training-Free Memory Module for LLMs*, 2026. https://github.com/PioneerQyw/NGM

Rasmussen, P., Paliychuk, P., Beauvais, T., Ryan, J., and Chalef, D. *Zep: A Temporal Knowledge Graph Architecture for Agent Memory*, 2025. https://arxiv.org/abs/2501.13956v1

Shin, S., Cha, M., Lee, B.-J., and Park, S. *ERA: Evidence-based Reliability Alignment for Honest Retrieval-Augmented Generation*, 2026. https://arxiv.org/abs/2604.20854v1

Wang, H., Luo, Y., and others. *MemTrapBench: Benchmarking Cognitive Traps in LLM Memory Use*, 2026. https://www.semanticscholar.org/paper/MemTrapBench%3A-Benchmarking-Cognitive-Traps-in-LLM-Wang-Luo/736d61825a5afed4c85b227951a9880d01e2299f

Xu, F. F., Alon, U., and Neubig, G. *Why do Nearest Neighbor Language Models Work?*, 2023. https://arxiv.org/abs/2301.02828v2

Xu, J., Shi, H., Shan, Y., Liu, P., Bai, Y., Li, N., and Liu, X. *TrustMargin: Training-Free Arbitration between Parametric Memory and Retrieved Evidence in Large Language Models*, 2026. https://arxiv.org/abs/2606.08397v1

Yu, Z., Zhao, Y., Cohan, A., and Zhang, X.-P. *HumanEval Pro and MBPP Pro: Evaluating Large Language Models on Self-invoking Code Generation*, 2024. https://arxiv.org/abs/2412.21199v2

Yu, Z., Xing, W., Wei, Y., Yang, B., Ye, C., Li, G., and Han, M. *The Attribution Blind Spot: Detecting When Language Models Rely on Memory Rather Than Retrieved Context*, 2026. https://arxiv.org/abs/2605.26778v1

Zhang, Z., Zeng, Z., Lin, Y., Wang, H., Ye, D., Xiao, C., Han, X., Liu, Z., Li, P., Sun, M., and Zhou, J. *Plug-and-Play Knowledge Injection for Pre-trained Language Models*, 2023. https://arxiv.org/abs/2305.17691v2

Zheng, Y., Wen, B., and Wang, X. *Lngram v2: Latent N-Gram Memory with Interpretable Discrete Representations*, 2026a. https://arxiv.org/abs/2609.03426v1

Zheng, Y., Xia, G., Wang, X., and Ren, L. *Lngram: N-gram Conditional Memory in Latent Space*, 2026b. https://arxiv.org/abs/2605.24869v1

Zhong, Z., Lei, T., and Chen, D. *Training Language Models with Memory Augmentation*, 2022. https://arxiv.org/abs/2205.12674v3

Zhou, W., Gu, Y., Iacovides, G., Qiu, Y., Zhao, Q., and Mandic, D. *Tensorizing Engram: Sharing Latents Across N-Gram Embeddings is Beneficial in LLMs*, 2026. https://arxiv.org/abs/2606.08347v1

A. Related Work, Extended

A.1 External Memory Layers

DeepSeek Engram (X. Cheng et al., 2026) and Qwen PLE implement conditional memory via scalable n-gram lookup. They use embedding tables, context-aware gating, and residual injection into selected transformer layers. Memory Grafting (R. Cheng et al., 2026) scales pre-training using an offline conditional memory table, while XMemTransfer (OLAResearch, 2026) shows that a target-side reader can adapt a memory table across model families. Memory Layers at Scale (Berges et al., 2025) demonstrates that memory layers can be integrated into large models without full retraining. Product-key memory layers (Lample et al., 2019) provide a related sparse lookup mechanism, and recent latent n-gram architectures such as Lngram v2 (Zheng et al., 2026a; 2026b) and Tensorizing Engram (Zhou et al., 2026) improve the parameter efficiency and interpretability of discrete memory addressing. In the small-model regime, PEMA (Kim et al., 2023), memory-augmented training (Zhong et al., 2022), Knowledge-in-Context (Pan et al., 2022), and plug-and-play knowledge injection (Zhang et al., 2023) show that external memory can be adapted after pretraining without full retraining.

A.2 Long-Term and Agent Memory

A parallel line of work treats memory as a first-class agent resource. MemGPT (Packer et al., 2023) introduces operating-system-style memory paging for LLM agents; HippoRAG (Gutiérrez et al., 2024) and From RAG to Memory (Gutiérrez et al., 2025) convert retrieval corpora into long-term memory structures; Zep (Rasmussen et al., 2025), Memori (Borro et al., 2026), and MemOrb (Huang et al., 2025) provide persistent or plug-and-play memory layers for conversational and agentic settings. These systems target long-range semantic memory, whereas our work focuses on a narrower, auditable token-level n-gram memory that is useful in low-entropy local continuations.

A.3 Non-Parametric Language Models

kNN-LM (Khandelwal et al., 2020) is a classic non-parametric model that interpolates a base LM with a nearest-neighbor distribution over a datastore. Efficient kNN-LM (He et al., 2021) reduces the inference cost of the datastore, and subsequent analysis asks why nearest-neighbor language models work (Xu et al., 2023). Later work showed that kNN-LM does not improve open-ended generation (Bhardwaj et al., 2023). NGM (PioneerQyw, 2026) provides a training-free n-gram memory hook. Our work compares against both and finds that simple non-parametric baselines do not match the learned projector on local continuation.

A.4 Distribution-Level Memory

MemSFT (Group, 2026) and TokenMem (T. Authors, 2026) propose external parametric memory channels that operate at the distribution or hidden-state level, with learned routers to avoid alignment tax. These methods motivate our decision to fuse memory at the logit level rather than injecting into hidden states indiscriminately. Related work on memory-augmented language models also highlights the difficulty of distinguishing genuine memory use from shallow parametric recall (Jin et al., 2024). Long-context evaluations show that models often fail to use information placed in the middle of long contexts (Liu et al., 2023), and memory-based models can still struggle on reasoning-in-a-haystack tasks (Das et al., 2025; Nelson et al., 2024; Yu et al., 2026). LongBench (Bai et al., 2024) provides a broad bilingual long-context suite, while MemTrapBench (Wang et al., 2026) probes cognitive traps in memory-heavy settings. Earlier experiments in our project found that hidden-state injection without careful orthogonalization and gating can produce large real-vs-control gaps that are not attributable to the memory content.

A.5 Retrieval Reliability and Calibration

Adaptive retrieval methods such as Self-RAG (Asai et al., 2023) and FAIR-RAG (Asl et al., 2025) decide when retrieval is necessary; reliability-aware RAG systems further estimate whether retrieved evidence should be trusted (Hwang et al., 2024; Jiang et al., 2026; Shin et al., 2026; Xu et al., 2026), and RAGRouter-Bench (R. Authors, 2026) benchmarks learned query-routing policies. Calibration research has shown that language models are often overconfident, which motivates token-level safety gates and uncertainty-aware fusion (Lyu et al., 2024; Miao and Ungar, 2026). On code tasks, localized calibrated uncertainty helps identify unreliable predictions (Gros and Devanbu, 2025). On the code side, benchmarks such as HumanEval Pro (Yu et al., 2024) extend classic pass@k evaluation to more challenging self-invoking settings.

A.6 Parameter-Efficient Adapters

Table 1. The window is the product of kernel size and key order, confirmed by perturbing one token at a time and observing which output positions respond. For every row, all lags inside the predicted window moved the output and all lags outside it moved it by exactly zero. Rows 1 and 2 are the two production geometries: kernel 4 with order-3 keys is Engram and Qwen3.8-Flash-Next, and a kernel of 1 — the convolution removed — is the ablation that leaves V4.1-Flash with the key order itself. Checked by Section 3.4 in the project's test suite.

kernel k	key order n	conv pad $(k-1)n$	predicted kn	measured
4	3	9	12	**12**
1	4	0	4	**4**
2	3	3	6	6
4	2	6	8	8
3	3	6	9	9
4	4	12	16	16

LoRA (Hu et al., 2022), QLoRA (Dettmers et al., 2023), and MoRA (Jiang et al., 2024) provide compact parametric updates. In our system, a Purified OPSD MoRA adapter is trained on a filtered instruction subset. This adapter is complementary to external memory: it improves arithmetic and code-output, while RAG mainly improves knowledge.

B. The Window, Measured: All Geometries

Each row is a separate instantiation of the official layer, perturbed one token at a time as described in §3.4.

C. Logit-Correction System: Method and Algorithms

C.1 Method

The projector is a small MLP $f_{\theta(h_t,m_t)} = (\alpha_t, \beta_t)$ over the last hidden state and a feature vector m_t carrying the matched n-gram order, base and memory entropy, the density ratio, both top-1 probabilities, memory/base agreement, and a task one-hot. Its final layer is zero-initialized so the projection begins as the identity. A logistic policy on the same features predicts whether fusion will help at the current token and disables it otherwise, which is what keeps open-ended generation from degrading. Per-task calibration supplies a scale, bias and temperature, and only projector and policy parameters are trained; the backbone and the memory table stay frozen.

The policy deserves a word on its information-theoretic status, since it reads hidden states and memory features and could be mistaken for a way around the bound. It cannot be. The policy only sets α_t and β_t to zero or leaves them alone, so the corrected distribution is either the projector's or the base model's; a gate that removes a contribution cannot add information that the contribution did not carry. Formally, p_{fused} is a function of h_t together with e_t and the policy's own deterministic features, so the chain in §3.2 applies unchanged, and the policy's role is to **reduce** the exponent rather than to supply anything new.

The projector produces a per-token logit correction without modifying the frozen backbone. The learned variables are α_t (memory trust) and β_t (support bias).

Algorithm 1 PLE Projector inference

1. Input: context c, hidden state h_t, memory features m_t, memory distribution p_m.
2. Compute $\alpha_t, \beta_t = f_{\theta(h_t,m_t)}$.
3. Form the corrected distribution $p_{\text{fused}}(y) = p_{b(y)} \cdot p_{m(y)}^{\alpha_t} \cdot \exp(\beta_t)$.
4. Evaluate the token policy $g_t \in \{0, 1\}$.
5. If $g_t = 0$, reset $\alpha_t \leftarrow 0$ and $\beta_t \leftarrow 0$.
6. Normalize and return p_{fused}.

The policy is a small binary classifier trained on the same memory features used by the projector. It decides whether the n-gram memory should contribute to the current token.

Table 2. Joint system mean answer log-probability, three seeds; higher is better. RAG carries knowledge, the adapter carries arithmetic and code-output, and PLE alone carries neither. Standard deviations across seeds are below 0.012 in every cell.

System	Knowledge	Arithmetic	Code-output
Base	−8.140	−7.365	−14.250
RAG	−6.748	−7.365	−14.250
PLE	−8.140	−7.492	−14.250
MoRA	−7.691	−7.114	−12.292
RAG+MoRA	−6.704	−7.114	−12.292
All	−6.704	−7.237	−12.292

Table 3. 10k local-continuation results, five seeds. NLL is lower-is-better.

Seed	Base NLL	Fixed NLL	Proj. NLL	Base hit	Fixed hit	Proj. hit
0	3.148	3.051	2.930	0.407	0.427	0.463
1	3.328	3.346	3.114	0.410	0.420	0.460
2	3.451	3.426	3.002	0.413	0.430	0.493
3	3.238	3.066	2.848	0.397	0.420	0.473
4	3.197	3.041	2.913	0.400	0.427	0.417

Table 4. kNN-LM and NGM baselines. Neither matches the learned projector.

Baseline	NLL	Hit	Note
Base	2.633	0.505	kNN eval set
kNN-LM	2.847	0.540	better hit, worse NLL
Base	2.521	0.550	NGM eval set
NGM	2.521	0.550	near-zero change

Algorithm 2 Token-level policy and safety gate

1. Train a logistic classifier on paired real/control observations.
2. At inference, compute features m_t from the current context.
3. Predict $g_t = P(\text{helpful} \mid m_t)$.
4. If $g_t < \tau$, disable PLE fusion and use base logits only.
5. Otherwise, apply the learned projector correction and continue decoding.

D. Logit-Correction System: Results

Per-seed 10k local-continuation results, the kNN-LM and NGM comparisons, and the sweep over n-gram order and memory-bank size.

E. System Architecture and Inference Pipeline

F. HumanEval, Judge and Case Studies

Under greedy decoding the base model passes HumanEval/16, 18, 23, 25, 32, 33, 38, 41, 43, 46 and 49; BM25+PLE passes HumanEval/0, 10, 27, 35 and 41. The only overlap is HumanEval/41, so BM25+PLE reaches four problems the base model

Table 5. Sensitivity over n-gram order and memory-bank size on code continuation. Gain is the NLL improvement of calibrated real fusion over the base model; all rows are single-seed exploratory sweeps, not pre-registered comparisons.

Setting	Real-vs-control gap	Calibrated fused gain
order 2	20.23	0.456
order 3	20.69	0.501
order 4	20.49	0.574
order 5	20.53	0.605
memory 40/80	22.68	1.408
memory 120/240	19.47	0.755
memory 300/600	20.28	0.210

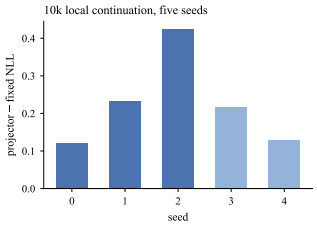

Figure 3. Per-seed projector-versus-fixed improvements on 10k data. Darker bars are seeds whose per-seed artifacts are retained in the released bundle and reproduce Table 3 exactly; the two lighter bars are reported from the original run, whose per-seed files were not retained.

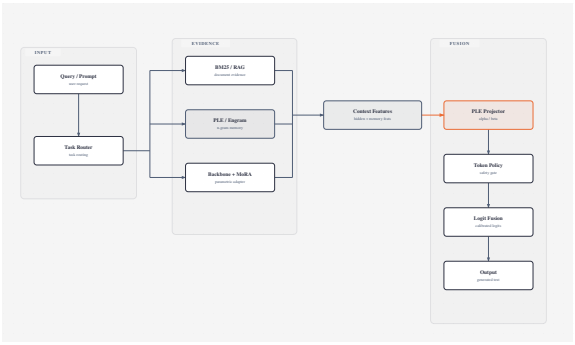

Figure 4. System architecture. The frozen backbone, RAG, and PLE memory provide three complementary evidence channels; the PLE Projector and token policy control logit-level fusion.

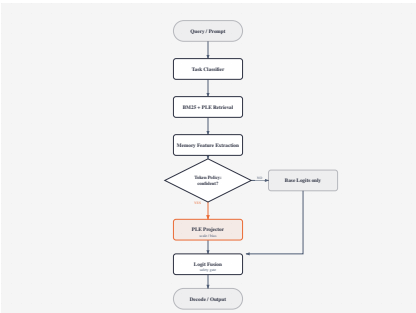

Figure 5. Inference pipeline. The token policy decides whether to apply the learned PLE Projector or bypass it with base logits only.

never solves — but it loses nine, and with no confidence interval over 50 problems we treat the recovery as suggestive rather than established.

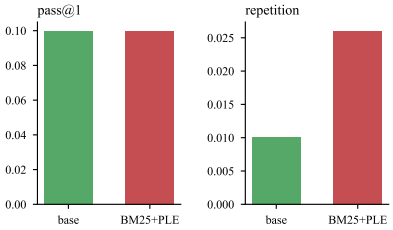

Figure 6. HumanEval pass@1 and mean repetition rate over 50 problems.

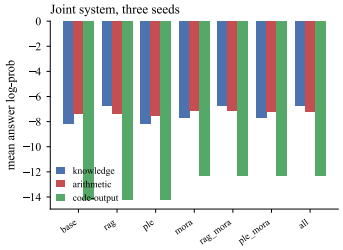

Figure 7. Joint system answer log-probability by resource, three seeds.

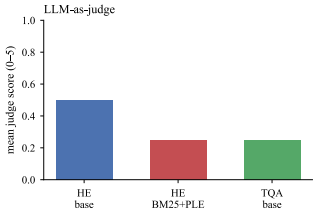

Figure 8. LLM-as-judge mean scores. Judge scores carry run-to-run variance and are reported as secondary evidence.

Table 6. Error analysis and mitigating mechanisms.

Failure mode	Observed evidence	Mitigation
Over-confident n-gram prior	Open-ended repetition and repository fragments	Learned token policy
Missing world knowledge	TriviaQA exact match = 0.005	RAG and parametric adapters
Hidden-state misalignment	Real-vs-control gains near zero	Logit-level calibrated fusion

TriviaQA failure. The frozen 0.8B model obtains 0.005 exact match on the 200-example validation subset. The failure is systematic rather than a calibration artifact: short-form knowledge questions require world knowledge that is not present in local n-gram continuations, and raw PLE fusion cannot recover it.

Open-ended degradation. Without the token-level policy, unconditional PLE fusion on natural-language code prompts produces repetitive fragments, unrelated repository text, and broken structure. The learned policy suppresses fusion on tokens where the n-gram prior is not clearly beneficial, which substantially reduces this degradation.

G. Method Audit: Full Table

H. Extended Limitations

1. The reader-fidelity check covers the read-out, not the addressing: we verify that our reader reproduces the official mathematics bit-for-bit at production geometry, but the table itself is accessed through our own row-identifier implementation, which is separately pinned against two independent implementations.
    1. The official Qwen3.8 model attaches its memory layer at 0-based layer 1, while our grafts inject at layer 2. The bound is a property of the reader's input-output map and is layer-independent, so the results above are unaffected, but the layer-1 configuration is untested rather than refuted.

Table 7. Every case found where a claim stood because its falsifying check was not run. The column that matters is the second: in each case the durable fix was making the failure detectable, and the detection is what we kept.

Unrun or skipped check	**How it surfaced**	**Claim it put at risk**
Row-identifier implementation differed from the proved-correct one	Cross-repo golden against two independent implementations	Every addressing claim
A convolution believed absent was in the executed path	Receptive-field measurement of the real layer (Section 3.4)	The twelve-token window
A loop body was indented so a metric scored 4 items, not 1500	Item-count fingerprint in the result manifest	The 0.8B saturated null
Four golden tests were skipping, not passing, on the compute box	Environment flag converting a silent skip into a failure	Every graft measurement
The reader's forward had never been compared to the official mathematics	Bit-exact golden at production geometry; they agree exactly	Every graft conclusion
A constant was copied from the wrong fixture (10^{-5} against 10^{-6})	The new golden test failed on first run	Nothing — caught before use
A skip flag was applied to a 65 MB asset absent from the repository	CI went red	Nothing — caught before use
A scheduled shutdown interrupted a queue with two arms on the data disk	Manifest whose arm count disagreed with the log	The 4B arm count, recovered on restart
A gold-NLL metric scored only the last batch	n-items-scored against the requested count	The table-probe numbers
An offline reproduction averaged over the diagonal of a similarity matrix	Unit test comparing it against the online statistic	The read-out collapse figure

2. Co-training the table with the backbone — the configuration all three production systems use, and the one most likely to change the value of the channel within the bound — is not tested here, at this scale.
3. The equivalence result in §4.1 is an exact agreement of empirical histograms over 1500 and 600 items, which rules out differences the sample can resolve — one item in n — and does not establish that the underlying distributions are identical. The equivalence margin is the sampling resolution, not a confidence interval on a difference.
4. The conclusion is scoped to frozen-table grafts onto a frozen backbone, injected at a single mid-stack layer, with a table of production size but a target model of 0.8B to 4B. We do not test co-training, learned routing, adaptive layer placement, or the memory-to-compute ratios the production systems use, and the bound is a statement about the channel rather than a prediction that those configurations are worthless.
5. Table size, memory bandwidth and serving cost are not evaluated here. The table we access is 47.7 GiB on disk; inference cost is dominated by row fetch rather than by arithmetic, and every arm in this paper is read from a local disk with no latency budget. Deployment trade-offs are therefore not addressed.
6. HumanEval is a public benchmark and we cannot rule out contamination of the base model's training data. The base model solves 11 of 50 problems greedily; if some of those are memorised, the comparison against BM25+PLE is affected in an unknown direction, which is a further reason we treat the code results as directional.
7. All experiments ran on a single NVIDIA RTX 4090 with 24 GiB and an AutoDL instance whose data disk held the table; software versions, container definitions, seeds, per-arm manifests and the analysis scripts are recorded with each result file, and the released bundle carries the raw evaluation outputs rather than summaries.
8. The bound is elementary — a data-processing inequality — and we do not claim otherwise. Its value is in the window arithmetic it forces on a family that three production systems scaled without writing it down, and in the experiment that tests its content prediction.

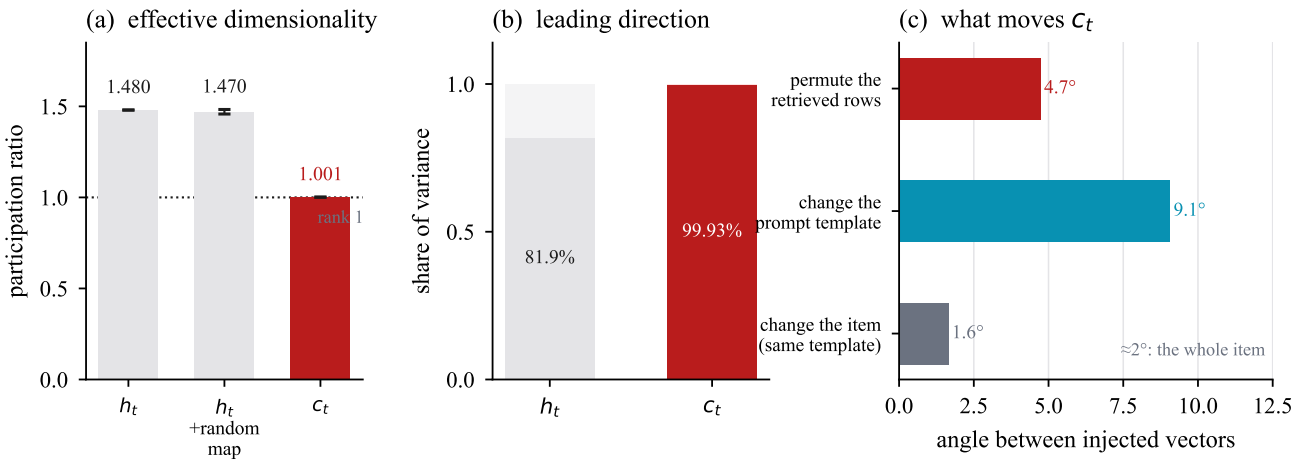

Figure 9. The read-out is close to a constant function of its input, which the ladder below localises. *(a)* Effective dimensionality of the injected vector, of the hidden state it is gated on, and of that hidden state after a random linear map of the same shape. Participation ratio $(\sum \lambda)^2 / \sum \lambda^2$ over $n = 600$ items; error bars on the random-map bar are one standard deviation across three maps, and bootstrap 95% intervals are $[1.0012, 1.0015]$ for c_t and $[1.433, 1.535]$ for h_t. *(b)* Share of variance in the leading direction. *(c)* What actually moves the injected vector.

9. Public projector and adapter weights are available on Hugging Face, but the upstream Qwen model weights themselves are not redistributed. CPU deployment throughput is about 2 tok/s with the current unoptimized eager implementation.

I. Real-vs-Control Protocol

To distinguish genuine memory use from model noise, we use a strict paired protocol.

1. The real memory is built from documents containing the target continuation.
2. The control memory is built from an unrelated document set with no target relation.
3. A method is considered useful only if it outperforms the control memory by a meaningful margin.
4. All projector results in the paper are paired across identical evaluation rows.

J. The Read-Out Collapse, Localised

§4.7 reports that the injected vector c_t has an effective dimensionality of 1.001 while the hidden state it is gated on sits at 1.480, and that the argument for calling this a read-out defect rested on a random map of h_t. That control bounds the measurement, not the explanation: the read-out's input is (h_t, e_t), and a faithful read-out fed a constant e_t would emit a constant too. Measuring the ladder at several positions back from the answer position varies e_t and so separates the two.

Each stage is the reader's own arithmetic, captured by forward hooks on `value_proj` and `out_proj`; the effective dimensionality is the participation ratio over the 600 evaluation items, and $\cos(\text{real}, \text{shuf})$ is the per-item cosine between the real and row-permuted conditions using the same permutation as the control arms.

Per task at the two informative offsets, addressed rows distinct out of 200 and the resulting c_t:

Two caveats. A participation ratio measures how many directions the variation spreads over, not how much information is present, so every claim here is about the shape of the variation rather than its quantity. And offset 12 is a different position from the one the arms read: it is used to characterise the read-out as a function, not to claim the arms read there.

Acknowledgement. We thank the open-source community for the PLE/Engram, Qwen, and related memory infrastructure used in this study. This work was carried out as an independent low-resource research project.

Table 8. The collapse is inherited from the addressing at the answer position and produced by the read-out everywhere else. At offset 0 every one of the 600 items addresses a byte-identical row, because all three prompt templates end in the same trigram, so a perfect read-out would still emit a constant. At offset 12 the addressed row is item-specific (190 of 200 BoolQ rows distinct, PR 136.7) and the same read-out still emits PR 1.02. Offset 40 is meaningful for BoolQ only: the NQ and TriviaQA prompts are shorter than 40 tokens, so those rows are clamped to position 0.

offset	stage	dim	PR	dup. rows	$\cos(\text{real}, \text{shuf})$
0	e_t	2560	1.000	99.7%	0.102
0	value proj	2560	1.000	98.7%	0.357
0	branch sum	2560	1.208	0.0%	0.619
0	c_t	1024	**1.001**	0.0%	**0.9966**
0	h_t	1024	1.480	0.0%	—
12	e_t	2560	**28.250**	27.0%	0.081
12	value proj	2560	14.387	26.2%	0.315
12	branch sum	2560	2.676	24.5%	0.711
12	c_t	1024	**1.019**	24.5%	**0.9958**
12	h_t	1024	3.717	24.5%	—
40	e_t	2560	3.379	66.5%	0.050
40	value proj	2560	2.521	66.2%	0.168
40	branch sum	2560	1.034	65.7%	0.358
40	c_t	1024	**1.012**	65.7%	**0.9927**
40	h_t	1024	2.453	65.7%	—

Table 9. Where the addressing is degenerate, so is the injection; where the addressing is rich, the injection is still degenerate.

offset	task	e_t distinct	c_t PR
0	BoolQ	1	1.158
0	NQ	1	1.001
0	TriviaQA	1	1.009
12	BoolQ	190	1.020
12	TriviaQA	172	1.015
12	NQ	79	1.030
40	BoolQ	198	1.011

Data Availability. All source code, evaluation scripts, container definitions, evaluation cards, and checksums are publicly available in the project repository and in the Hugging Face artifact repository. The released artifact bundle includes PLE projector checkpoints, Purified OPSD MoRA adapters, projector datasets, configurations, and the raw evaluation result files, including the per-item injected vectors behind the read-out measurement. The PLE memory tables can be rebuilt from the published corpus construction scripts, and upstream Qwen model weights are not redistributed.